D2 — Attack Surface and Working Harness

1. Attack surface analysis (Track B)

Entry points in libheif that parse untrusted input. All are reachable from the harnessed decode entry point.

#	Entry point	Input it parses	Reachable from harness?
1	ISOBMFF box parser	Box headers and sizes	Yes (file_fuzzer)
2	Item/property parser	Item references and properties	Yes (api_fuzzer)
3	Image handle decode	Pixel data and metadata	Yes (file_fuzzer)
4	Color conversion	Color profiles and matrices	Yes (color_conversion_fuzzer)
5	Tile grid decode	Tile offsets and sizes	Yes (tile_fuzzer)
6	Sequence/track decode	Track boxes and samples	Yes (sequence_fuzzer)
7	Box-level parser	Individual boxes	Yes (box_fuzzer)
8	Encoder path	Encoding parameters	Build only (encoder_fuzzer)

2. Programs and test cases

Program	Purpose	Build status
file_fuzzer	Decode a HEIF/HEIC/AVIF file	Builds
api_fuzzer	Exercise extended API surfaces	Builds
box_fuzzer	Parse individual boxes	Builds
color_conversion_fuzzer	Color conversion paths	Builds
tile_fuzzer	Tile grid decoding	Builds
sequence_fuzzer	Image sequences	Builds
encoder_fuzzer	Encoding paths	Builds

No fuzzer failed to build; all use the upstream targets unchanged.

3. Repository

https://github.com/lordzadd/libheif-fuzz-d2

4. Exact commands to run the harness

```
git clone https://github.com/lordzadd/libheif-fuzz-d2.git
cd libheif-fuzz-d2
python3 harness/run.py
```

Optional:

```
python3 harness/run.py
```

The harness prints a JSON summary with `total_crashes` and writes `results/fuzz_results.csv`.

AI Use Appendix

I used an AI coding assistant to draft the harness, build script, and this document. I reviewed and edited the output and tested the structure locally. EOF